SCA101 - LAB 0 TO LAB 2_1A

# Linux Installation and Setup Guide

Install ChipWhisperer on Linux, wire up the Husky kit, and work through Lab 0 and Lab 2_1A

A step-by-step walk-through of the Linux install, with annotated pictures of how to connect the hardware and the fixes for the problems people actually run into. Written so that nobody needs prior Linux experience: every step says what to type, what you should see, and what to try if you do not see it. It runs all the way through **Lab 2_1A**, including the hands-on **Explorer** notebook.

> **HOW TO USE THIS GUIDE**
> Work from top to bottom and **do not skip the checks**. If a step goes wrong, go straight to **Troubleshooting** (Part 6): each entry starts with the exact message you will see on screen.

## The three links you need

| What | Where |
|---|---|
| NewAE's own Linux install page (the steps in Part 1 come from here) | https://chipwhisperer.readthedocs.io/en/latest/linux-install.html |
| The course notebooks, branch `sca101-lab0-lab2-complete` (Part 2) | https://github.com/LFB1937/chipwhisperer-jupyter |
| The ChipWhisperer software and firmware, commit `7b3206b0` (Part 1) | https://github.com/newaetech/chipwhisperer |

## The four colored boxes you will see everywhere

**DO THIS**
An action to take.

**YOU SHOULD SEE**
What the screen should show.

**IF IT DOESN'T**
What to try if it didn't.

**WATCH OUT**
A mistake that costs time.

Guide version 1.5 - November 2026. Tested on **Ubuntu 22.04 LTS (Python 3.10, WSL2)**: the install, Lab 0, Lab 2_1A in simulated mode, the Explorer notebooks and the firmware builds. Tested on **Ubuntu 24.04 LTS (Python 3.12) in a VMware Workstation 17 Player VM with a ChipWhisperer-Husky**: the install, udev rules, USB pass-through, Jupyter, the scope connection and programming the SAM4S target. A fresh Python 3.12 install with all the simulated notebooks was also tested (x86-64). The hardware pictures are annotated from NewAE's own photos. The **Source** line under each troubleshooting entry says where it comes from.

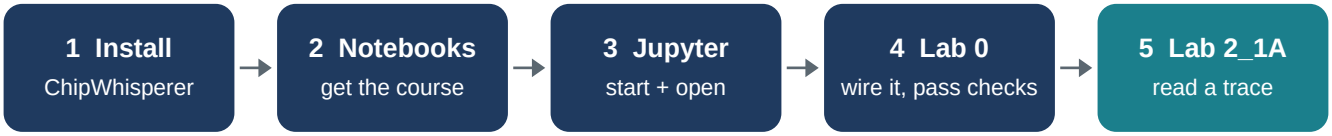
# Contents

**The big picture . . . 3**
**Before you start . . . 5**
**Part 1 - Install ChipWhisperer . . . 7**
1.1 Install the system packages . . . 7
1.2 Download ChipWhisperer . . . 8
1.3 Create and activate a virtual environment . . . 8
1.4 Give your user permission to use the USB device . . . 9
1.5 Install ChipWhisperer into the environment . . . 9
1.6 Log out and back in (or reboot) . . . 10
**Part 2 - Get the course notebooks . . . 10**
2.1 Put the notebooks in the right place . . . 10
2.2 Install the notebook requirements . . . 11
**Part 3 - Start Jupyter and open Lab 0 . . . 12**
3.1 Start Jupyter . . . 12
3.2 Open Lab 0 . . . 12
3.3 Check the notebook is using your environment . . . 12
**Part 4 - Lab 0: wire it up and check it . . . 13**
4.1 How the labs are organised . . . 13
4.2 Connect the Husky kit (skip this if you are only doing SIMULATED labs) . . . 14
4.3 Section by section . . . 21
4.4 What you should be able to do at the end of Lab 0 . . . 21
**Part 5 - Lab 2_1A: Instruction Power Differences . . . 22**
5.1 Open Lab 2_1A (MAIN) and choose the version . . . 22
5.2 Run it with recorded traces (SIMULATED) . . . 23
5.3 Optional: run it on the real chip (HARDWARE) . . . 24
5.4 Do the Explorer notebook . . . 25
5.5 What you should be able to do at the end of Lab 2_1A . . . 26
**Part 6 - Troubleshooting . . . 27**
6.1 Installing packages (apt, venv, pip) . . . 27
6.2 Jupyter and notebook kernels . . . 30
6.3 USB devices and permissions . . . 31
6.4 Compilers and firmware builds . . . 33
6.5 Virtual machines . . . 34
6.6 The Husky kit (any operating system) . . . 35
6.7 Lab 2_1A and the Explorers . . . 35
6.8 Still stuck? Collect this and ask for help . . . 37
**You are done when... . . . 37**
**Appendix A - Quick reference . . . 38**
**Appendix B - Glossary . . . 40**
**Appendix C - Where things live . . . 40**

# The big picture

**What are we doing?** The SCA101 labs are Jupyter notebooks that talk to a ChipWhisperer capture board over USB, build firmware for the target chip, and analyse the power traces. Before any of that works your Linux machine needs four things: the ChipWhisperer Python software, a compiler for the target firmware, permission for your normal user account to open the USB device, and the notebooks themselves. Lab 0 is the test that proves all four are in place, and Lab 2_1A is your first real use of them.

### Your path through the guide:




| Part | What you do | You are done when... |
|---|---|---|
| **1 Install ChipWhisperer** | Install packages, download ChipWhisperer, make a Python virtual environment, set up USB permissions. | `import chipwhisperer` works inside the virtual environment. |
| **2 Get the notebooks** | Download the course notebooks (the `chipwhisperer-jupyter` repository) into the right place and install their Python requirements. | `jupyter` and the notebook folders exist. |
| **3 Start Jupyter** | Launch Jupyter, find the SCA101 folder, open Lab 0. | Lab 0 is open and its kernel uses your virtual environment. |
| **4 Run Lab 0** | Connect the Husky kit (with pictures), then work through the notebook and read the results. | Every check passes (see 4.4). |
| **5 Run Lab 2_1A** | Capture or replay four power traces, then explore them in the Explorer notebook. | You can read a loop in a power trace (see 5.5). |
| **6 Troubleshooting** | Look up your error message. | Your problem is fixed. |

### What gets installed, and why

| Piece | Why you need it |
|---|---|
| `make`, `git` | Build target firmware; download the code. |
| `gcc-arm-none-eabi` + `libnewlib-arm-none-eabi` | Compile firmware for the Arm targets (the SAM4S chip in the Husky kit, and STM32). |
| `gcc-avr`, `avr-libc` | Compile firmware for the AVR/XMEGA targets. Not used for the SAM4S, but it is on NewAE's list. |
| `libusb-1.0-0-dev`, `usbutils` | The USB library ChipWhisperer talks through, and the `lsusb` tool you will use to debug. |
| `python3`, `python3-venv`, `python3-dev` | Python itself, private environments (`venv`), and headers for packages that build code. |
| `50-newae.rules` and the `chipwhisperer` group | Lets your normal user account open NewAE USB devices without `sudo`. |
| The Python packages | ChipWhisperer itself, Jupyter, and the plotting and analysis libraries the labs use. |
| `cwtraces` | The recorded power traces the SIMULATED labs replay (Lab 2_1B and later). **Not** installed by the standard steps - see step 2.2. |

# Before you start

Tick every row before Part 1. Most problems later come from something on this list.

| | You need | How to check |
|---|---|---|
| [ ] | A Linux machine or virtual machine running **Ubuntu or Debian** (Ubuntu 22.04 LTS and 24.04 LTS are the ones this guide was tested on). | `cat /etc/os-release \| head -2` |
| [ ] | **Python 3.9 to 3.12.** Python 3.13 or newer will not install (see the warning below). | `python3 --version` |
| [ ] | An account with `sudo` rights and a working internet connection. | `sudo -v` asks for your password and prints nothing. |
| [ ] | **Optional:** the Husky kit (ChipWhisperer-Husky, CW313 base board, CW312T-SAM4S target, cables) and a USB-C **data** cable. | Only needed for the HARDWARE labs. Without it you can still do almost everything: the SIMULATED labs use recorded traces. |
| [ ] | **If you use a virtual machine:** USB pass-through set up for NewAE devices. | See Troubleshooting 6.5 before you plug anything in. |

### Which Linux versions are covered?

| Distribution | Status |
|---|---|
| Ubuntu 22.04 LTS (Python 3.10), in WSL2 | **Tested** - every install command, the Lab 0 software checks, Lab 2_1A (simulated), the Explorer notebooks and the firmware builds were run here. No USB hardware (WSL). |
| Ubuntu 24.04 LTS (Python 3.12), in a VMware Workstation 17 Player VM with a Husky | **Tested** - the install, udev rules, USB pass-through, Jupyter, the scope connection and SAM4S programming. These systems refuse `pip install` outside a virtual environment, which is one more reason this guide uses one (see L-3). |
| Python 3.12 on x86-64, fresh install (what Ubuntu 24.04 uses) | **Tested** - the install commands and all six simulated notebooks (Lab 0, Lab 2_1A and its Explorer, Lab 2_1B and its Explorer, the solved Lab 2_1B) ran with no errors. **This needs the numpy limit in step 2.2** - without it the install silently breaks (L-49). |
| Ubuntu 24.04 LTS on ARM64 (aarch64) computers | Step 1.1 was confirmed on one such desktop. Every Python package the install needs has an ARM64 build for Python 3.12 (checked by resolving the install for ARM64), apart from `fastdtw`, which compiles from source or falls back to pure Python (L-5). The remaining steps have **not** been confirmed on ARM64 yet. |
| Debian 12 (Python 3.11) | Expected to work with the same commands (not tested). |
| Anything with Python 3.13 or newer (for example Debian 13, Ubuntu 25.04 and later) | Needs a Python 3.12 virtual environment - see L-4. |
| Fedora, Arch and others | The steps are the same but the package names differ. Use the NewAE page as a guide. |

**COURSE VERSIONS**

**Pinned versions.** This guide installs one specific ChipWhisperer commit, **7b3206b0** (the 6.0.0 software the labs were validated on), and the course notebooks from branch **sca101-lab0-lab2-complete**. NewAE's current `develop` branch is already 7.0.0: it needs Python 3.11 or newer and numpy 2, and has not been tested with these labs or with Husky firmware 1.5.0. Use the exact commands in Parts 1 and 2 - do not skip the `git checkout` step.

**WATCH OUT**

**Python 3.13 or newer will fail.** ChipWhisperer pins `numpy<=1.26.4`, and numpy 1.26.4 has no pre-built package for Python 3.13 or newer. Check `python3 --version` now. If it says 3.13 or higher, read entry **L-4** in Troubleshooting before continuing.

# Part 1 - Install ChipWhisperer

NewAE publishes the official steps at https://chipwhisperer.readthedocs.io/en/latest/linux-install.html. That page has a **Quick Installation** and a **Manual Installation** section that contain the same commands. This guide follows the Quick Installation, split into small steps so you can see exactly where something goes wrong. Things that are **not** on NewAE's page are marked **[added]**.

> **GOOD TO KNOW**
>
> **Open one terminal and keep using it.** Every command in this guide is typed into a terminal (on Ubuntu: press `Ctrl+Alt+T`). Lines starting with `sudo` ask for your password; nothing appears on screen while you type it. That is normal.

> **COPYING COMMANDS FROM THIS PDF**
>
> **Copy and paste ONE line at a time.** Many PDF viewers join the lines of a code box into a single line when you copy it, so a command that is split over several lines can arrive broken. (A trailing backslash becomes `\ ` - a backslash and a space - and the shell then glues a space onto the next word; this produces errors such as `E: Unable to locate package libnewlib-arm-none-eabi`, with two spaces. See L-48.) For that reason every command in this guide is on a **single line**, and each box below holds separate commands. Paste one line, press **Enter**, wait for it to finish, then paste the next. Typing the command works just as well.

## 1.1 Install the system packages

```
sudo apt update && sudo apt upgrade
```

```
sudo apt install make git avr-libc gcc-avr gcc-arm-none-eabi libnewlib-arm-none-eabi
```

```
sudo apt install libusb-1.0-0-dev usbutils python3 python3-venv python3-dev
```

`libnewlib-arm-none-eabi` is **[added]**. apt normally installs it automatically alongside `gcc-arm-none-eabi`, but naming it makes the install work even on machines where apt is set to skip recommended packages. (The `apt install` list is split over two commands only so that each fits on one line.)

> **YOU SHOULD SEE**
>
> All three commands finish without any line starting with `E:`. Then check the two tools the labs need:
>
> ```
> make --version | head -1
> arm-none-eabi-gcc -dumpversion
> ```
>
> On Ubuntu 22.04 this printed `GNU Make 4.3` and `10.3.1`. Your numbers may differ; what matters is that both commands print a version instead of `command not found`.

> **IF IT DOESN'T**
>
> `E: Unable to locate package ...` - if the name has an extra space in front of it, you pasted a multi-line command (L-48); otherwise run `sudo apt update` first (L-1). For anything else, see Troubleshooting 6.1 and 6.4.

## 1.2 Download ChipWhisperer

```
cd ~/
git clone https://github.com/newaetech/chipwhisperer
cd chipwhisperer
git checkout 7b3206b0
```

The last line (**[added]**) switches to the exact version the labs were validated on. Git prints a long message about a **detached HEAD** - that is normal; it just means you are looking at one fixed version rather than a branch.

**YOU SHOULD SEE**

`ls` lists `firmware`, `software`, `docs`, `pyproject.toml`, `50-newae.rules` and more, and `git log -1 --oneline` prints `7b3206b0 Add note for CW506 IO levels`. You are now inside `~/chipwhisperer`. Stay in this folder for the next steps.

**IF IT DOESN'T**

`Could not resolve host: github.com` means the machine has no internet (L-9). `git: command not found` means step 1.1 did not finish.

**WHERE AM I?**

**Where am I?** The end of your terminal prompt shows the folder you are in. `~` means your **home** folder; `~/chipwhisperer` means you are **inside the ChipWhisperer folder**. Steps 1.4, 1.5 and several later ones only work from `~/chipwhisperer`. A **new terminal window always starts in** `~`, and `cd ..` takes you back up a level - so whenever a step tells you to `cd ~/chipwhisperer`, do it. If you are lost, type `pwd` (it prints the full folder name) or just run `cd ~/chipwhisperer`. See L-50.

## 1.3 Create and activate a virtual environment

A **virtual environment** is a private copy of Python for ChipWhisperer, so its packages never clash with anything else on your system. NewAE recommends one, and newer Linux versions require it.

```
python3 -m venv ~/.cwvenv
source ~/.cwvenv/bin/activate
```

**YOU SHOULD SEE**

Your prompt now starts with `(.cwvenv)`. `which python` prints `/home/<you>/.cwvenv/bin/python`.

**WATCH OUT**

**Activate it again in every new terminal** with `source ~/.cwvenv/bin/activate`. If the prompt does not start with `(.cwvenv)`, you are not inside the environment and the next commands will fail or install into the wrong place (L-3, L-11, L-12).

## 1.4 Give your user permission to use the USB device

Linux normally lets only `root` open raw USB devices. NewAE ships a rules file that hands NewAE devices to a group called `chipwhisperer`; you then put yourself in that group. **First go back into the ChipWhisperer folder** - the rules file is in there, and the first command below fails if you are anywhere else (your prompt should end in `~/chipwhisperer`):

```
cd ~/chipwhisperer
```

```
sudo cp 50-newae.rules /etc/udev/rules.d/50-newae.rules
sudo udevadm control --reload-rules
sudo groupadd -fr chipwhisperer
sudo usermod -aG chipwhisperer $USER
sudo usermod -aG plugdev $USER
```

| Command | What it does |
|---|---|
| `sudo cp 50-newae.rules ...` | Installs the rules. Any USB device with vendor ID `2b3e` (NewAE) becomes readable and writable by group `chipwhisperer`, and serial ports appear as `/dev/cw_serial*` and `/dev/cw_bootloader*`. |
| `sudo udevadm control --reload-rules` | Tells Linux to re-read the rules. They apply to a device the **next time it is plugged in**. |
| `sudo groupadd -fr chipwhisperer` | Creates the group (and does nothing if it already exists). Newer systemd versions need a system group for udev. |
| `sudo usermod -aG chipwhisperer $USER` | Adds you to the group. The `-a` matters: without it you would be **removed** from your other groups. |
| `sudo usermod -aG plugdev $USER` | Older instructions used this group. Harmless; if your distribution has no `plugdev` group you will see an error you can ignore (L-7). |

**IF IT DOESN'T**

`cp: cannot stat '50-newae.rules': No such file or directory` - you are not inside `~/chipwhisperer` (or step 1.2 never finished). Run `cd ~/chipwhisperer`, check that `ls 50-newae.rules` prints the file name, and try again (L-50).

**WATCH OUT**

Group membership is only read **when you log in**. You will do the log-out in step 1.6 - until then `id` will not list `chipwhisperer` and Python will not be able to open the device.

## 1.5 Install ChipWhisperer into the environment

```
cd ~/chipwhisperer
```

```
python -m pip install -e .
```

Run this from inside `~/chipwhisperer` with `(.cwvenv)` showing. `-e` means **editable**: Python uses the code in this folder directly, so `git pull` later updates it without reinstalling.

**YOU SHOULD SEE**

It takes about 30 seconds and ends with a line like `Successfully installed ... chipwhisperer-6.0.0 ... numpy-1.26.4 ...` (versions may differ). You may also see `UserWarning: compilation failed. Installing pure python package`. That is **harmless** (L-5). Then test:

```
python -c "import chipwhisperer as cw; print(cw.__version__)"
```

It prints `6.0.0` and no error. (If it prints `7.0.0` you skipped the `git checkout` in step 1.2 - see L-34.)

**IF IT DOESN'T**

`error: externally-managed-environment` - the environment is not active (L-3). `Failed building wheel for numpy` - your Python is too new (L-4).

### 1.6 Log out and back in (or reboot)

This is what makes the new group membership real. NewAE's page says to restart; logging out and in again is enough, but a reboot is the surest. On a virtual machine, restart the VM. Then open a terminal and check:

```
id
ls -l /etc/udev/rules.d/50-newae.rules
```

**YOU SHOULD SEE**

The output of `id` contains `chipwhisperer` in the list of groups, and the rules file exists. If you have a ChipWhisperer, **unplug it and plug it back in now**, then run `lsusb | grep -i 2b3e`. A line mentioning NewAE or ChipWhisperer means Linux can see it (for a Husky the ID is `2b3e:ace5`).

# Part 2 - Get the course notebooks

The labs live in a separate repository called **chipwhisperer-jupyter**. This guide uses the course branch **sca101-lab0-lab2-complete** of https://github.com/LFB1937/chipwhisperer-jupyter, which holds the finished Lab 0, Lab 2_1A and Lab 2_1B notebooks and the new Explorer notebooks. (NewAE's own copy is at https://github.com/newaetech/chipwhisperer-jupyter.) Clone it, then install the extra Python packages the notebooks need.

### 2.1 Put the notebooks in the right place

The hardware labs build firmware from the ChipWhisperer repository. They find it using a relative path (`../../../firmware/...`), so the notebooks must sit **inside** the `chipwhisperer` folder, in a folder called `jupyter`, next to `firmware`:

```
~/chipwhisperer/
    firmware/                  <- from the chipwhisperer repository
    software/
    jupyter/                   <- the chipwhisperer-jupyter repository goes HERE
        courses/
            sca101/
                Lab 0 - SCA101 Setup.ipynb
                Lab 2_1A - Instruction Power Differences (MAIN).ipynb
                Lab 2_1A - Instruction Power Explorer.ipynb
                traces/
        requirements.txt
```

Do this from inside `~/chipwhisperer`. It clones the course branch straight into a folder called `jupyter`:

```
cd ~/chipwhisperer
```

```
git clone -b sca101-lab0-lab2-complete https://github.com/LFB1937/chipwhisperer-jupyter jupyter
```

**GOOD TO KNOW**

**Why not NewAE's `git submodule update --init jupyter`?** At the pinned ChipWhisperer commit that fetches an older copy of the labs: Lab 2_1A is still the blank worksheet (no `VERSION` switch) and the recorded traces it needs are missing. The course branch has the finished labs.

**WATCH OUT**

**Do not clone `chipwhisperer-jupyter` on its own into your home folder.** The notebooks would then look for `../../../firmware` somewhere that does not exist, and every HARDWARE lab fails at the build step (L-26). The `SIMULATED` labs would still run, which makes this mistake easy to miss.

**YOU SHOULD SEE**

`ls jupyter/courses/sca101` lists `Lab 0 - SCA101 Setup.ipynb`, the `Lab 2_1A ...` files including `Lab 2_1A - Instruction Power Explorer.ipynb`, the `Lab 2_1B ...` files, and a `traces` folder.

**IF IT DOESN'T**

`fatal: destination path 'jupyter' already exists and is not an empty directory` - you cloned it before; see L-8.

## 2.2 Install the notebook requirements

```
cd ~/chipwhisperer
```

```
python -m pip install -r jupyter/requirements.txt cwtraces "numpy<=1.26.4"
```

```
python -c "import chipwhisperer as cw, numpy; print(cw.__version__, numpy.__version__)"
```

The **install command** puts three things into one pip run: Jupyter and the plotting and analysis libraries (bokeh, holoviews, pandas, matplotlib, ipywidgets and more); `cwtraces` (**[added]**), the package of recorded power traces that the SIMULATED labs replay - it is **not** in `requirements.txt`, so without it Lab 2_1B and the later simulated labs stop with `ModuleNotFoundError: No module named 'cwtraces'` (L-47); and the

**numpy limit** `"numpy<=1.26.4"` (**[added]**, and important - see the box below). Type the quotation marks exactly as shown. The last command is a **check**.

**WHY THE NUMPY LIMIT?**

ChipWhisperer 6.0.0 needs **numpy 1.26.4 or older**. On Python 3.12 (the default on Ubuntu 24.04) pip would otherwise pick the newest `scipy`, which needs **numpy 2 or newer**. Faced with that conflict, pip does not warn you - it **silently replaces your ChipWhisperer 6.0.0 with the old 5.6.1** from the internet and installs numpy 2. Everything seems to work until Lab 2_1B fails with `TypeError: Chart.__init__() missing 1 required positional argument: 'data'`. The numpy limit makes pip choose an older `scipy` (1.17) instead and keep ChipWhisperer. Reproduced on a clean Python 3.12 install; see L-49.

**YOU SHOULD SEE**

The install takes about two minutes on a fast connection and downloads roughly a hundred and twenty packages; it ends with `Successfully installed ...` and a very long list. **Do not press** `Ctrl+C` while it looks busy. The check prints exactly `6.0.0 1.26.4`. If the first number is not `6.0.0`, or the second does not start with `1.`, stop and read L-49.

**GOOD TO KNOW**

**Order matters.** `requirements.txt` includes `chipwhisperer` itself. Because you already ran `pip install -e .` in step 1.5, pip is meant to keep your editable copy. If you reverse the order you get the released version from PyPI instead of the code in `~/chipwhisperer`. Even in the right order you still need the numpy limit.

# Part 3 - Start Jupyter and open Lab 0

## 3.1 Start Jupyter

```
source ~/.cwvenv/bin/activate
cd ~/chipwhisperer/jupyter
jupyter notebook
```

**YOU SHOULD SEE**

The terminal prints a few lines including a web address such as `http://localhost:8888/tree?token=...`, and your browser opens on a page listing folders. **Leave this terminal open** - closing it stops Jupyter. (If no browser opens, copy that whole address into one; see L-13.)

## 3.2 Open Lab 0

1. In the browser file list, click **courses**.
2. Click **sca101**.
3. Click **Lab 0 - SCA101 Setup.ipynb**. It opens in a new tab.

## 3.3 Check the notebook is using your environment

This one check prevents the most common Jupyter problem. Click into an empty cell (or use **Insert > Insert Cell Below**), type the lines below and press **Shift+Enter**:

```
import sys
print(sys.executable)
```

**YOU SHOULD SEE**

The path printed contains `.cwvenv`, for example `/home/<you>/.cwvenv/bin/python`. Delete that cell afterwards.

**IF IT DOESN'T**

A path like `/usr/bin/python3` means Jupyter was started without the environment active (L-12). Stop Jupyter with `Ctrl+C` in its terminal, activate the environment, start again.

### Every time you come back

The install is a one-time job. Each later session is just:

```
source ~/.cwvenv/bin/activate
cd ~/chipwhisperer/jupyter
jupyter notebook
```

# Part 4 - Lab 0: wire it up and check it

**Lab 0 - SCA101 Setup** is the one-time setup check. You work through it once; every later lab assumes it passed. It teaches you how the labs are organised, how to set up the hardware, and proves your software works.

**GOOD TO KNOW**

**Older copy of the notebooks?** If your Lab 0 is short and only links to `ChipWhisperer Setup Test` and `1 - Connecting to Hardware` (both in the top `jupyter` folder), you have an older version. Work through those two notebooks instead; the pass conditions in 4.4 are the same.

## 4.1 How the labs are organised

Each lab comes as three files. You only ever open the first one.

| File | What it is |
|---|---|
| `... (MAIN).ipynb` | The lab itself. Runs top to bottom and has a `VERSION` switch near the top. **Open this one.** |
| `... (SIMULATED).ipynb` | Back-end that uses pre-recorded traces - no hardware needed. The MAIN notebook runs it for you. |
| `... (HARDWARE).ipynb` | Back-end that talks to your ChipWhisperer. The MAIN notebook runs it for you. |

In the MAIN notebook you set `VERSION = 'SIMULATED'` or `VERSION = 'HARDWARE'`. You do not need to open the other two files.

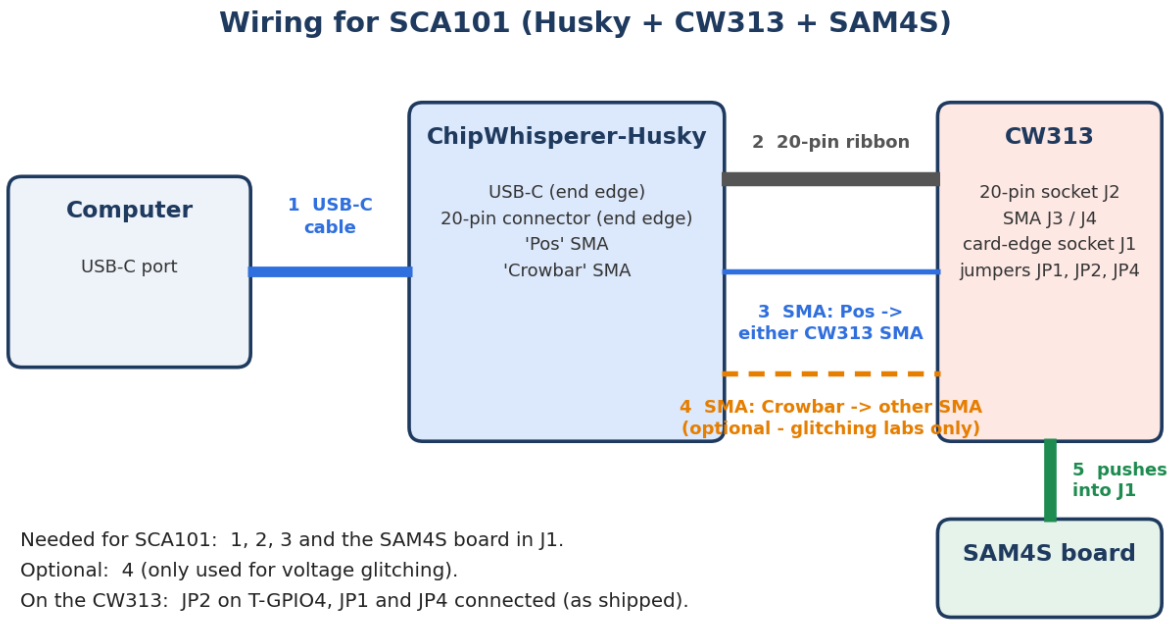
## 4.2 Connect the Husky kit (skip this if you are only doing SIMULATED labs)

This is where most hardware problems start, so take it slowly. You will connect **four things**: the SAM4S target board into the CW313, a ribbon cable between the Husky and the CW313, one SMA cable for the power measurement, and the USB cable. The pictures are annotated from NewAE's own photos of the same kit; your boards should look the same.

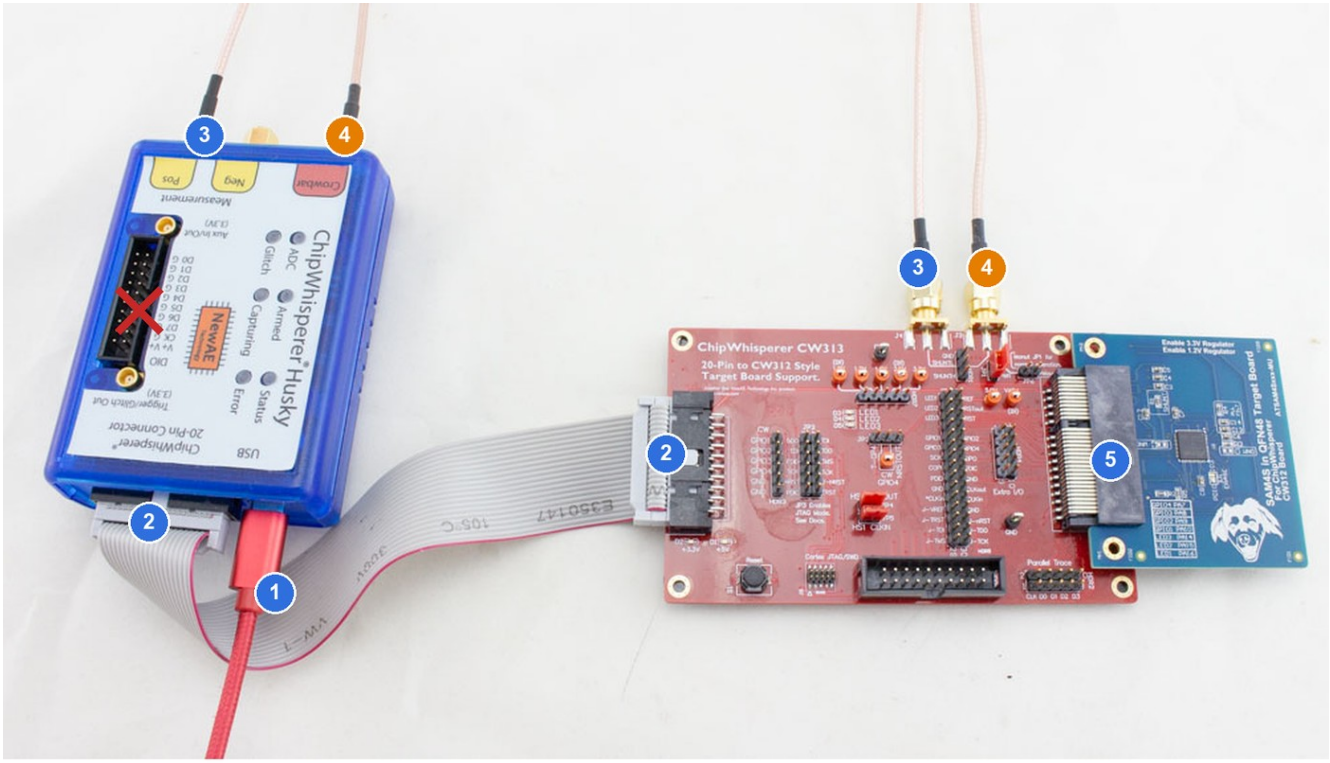
### The cables and boards in the kit

| Part | Used in SCA101? | What it connects |
|---|---|---|
| USB-C cable (blue) | **Yes** | Husky USB-C port to your computer. A USB-C to USB-A adapter is included for computers without USB-C. |
| 20-pin ribbon cable (two in the kit - use one) | **Yes** | Husky's 20-pin **edge** connector to the CW313's black 20-pin socket (J2). |
| SMA-to-SMA cable, 12 inch (two in the kit) | **One** (Pos) - the second is optional | Husky `Pos` to one CW313 SMA socket (power measurement). The second cable, Husky `Crowbar` to the other SMA socket, is for the voltage-glitching labs only. |
| CW312T-SAM4S target board | **Yes** | Pushes into the CW313's card-edge socket (J1). |
| CW312T-iCE40 target board | No | The FPGA target - not used in SCA101. |
| BNC-MCX cable, SMA-MCX cable, 20-pin breakout cable | No | For other labs. |

Cable list from NewAE's Husky starter-kit documentation; your kit may differ slightly.




The connections for SCA101 at a glance. Items 1, 2, 3 and the SAM4S board are needed; 4 is optional.




## Husky + CW313 + SAM4S, as in NewAE's own photo

1. USB-C cable from your computer into the Husky's USB-C port (on the same end as the ribbon connector).
2. 20-pin ribbon cable: from the Husky's edge connector labelled 'ChipWhisperer 20-Pin Connector' (next to USB-C) to the CW313's black 20-pin socket on the left.
3. SMA cable from the Husky's 'Pos' (Measurement) SMA to one of the two SMA sockets on top of the CW313. This is the power-measurement cable - SCA101 needs it.
4. SMA cable from the Husky's red 'Crowbar' SMA to the other CW313 SMA. NewAE's photo has it connected; it is only used for the voltage-glitching labs.
5. The SAM4S target board is pushed into the CW313's card-edge socket on the right (blue board, bear logo).

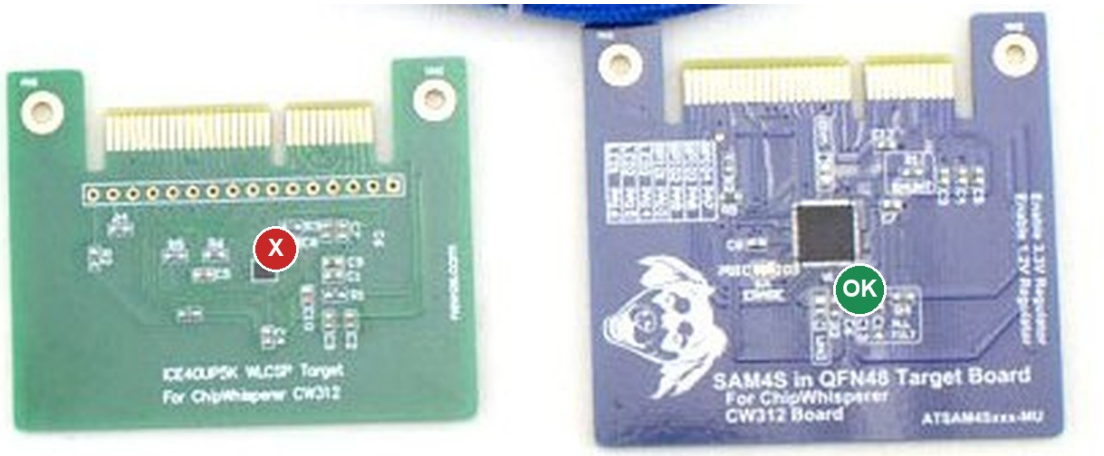
X Red cross: the Husky's central black header is the User IO header - NOT where the ribbon goes.

*Source: NewAE's photo jupyter/img/cwhusky.jpg, annotated. The two CW313 SMA sockets are wired together, so cables 3 and 4 can go to either socket (docs/source/Targets/CW313.md).*

NewAE's own photo of the finished setup, annotated. Use it to check your result at the end.

### Step 1 - Pick the right target board and seat it

The kit has **two** target boards that look alike. Use the one whose printed text says **SAM4S in QFN48 Target Board**. Mounting the iCE40 board by mistake is a known cause of `timeout` errors when the lab tries to program the chip (H-1).

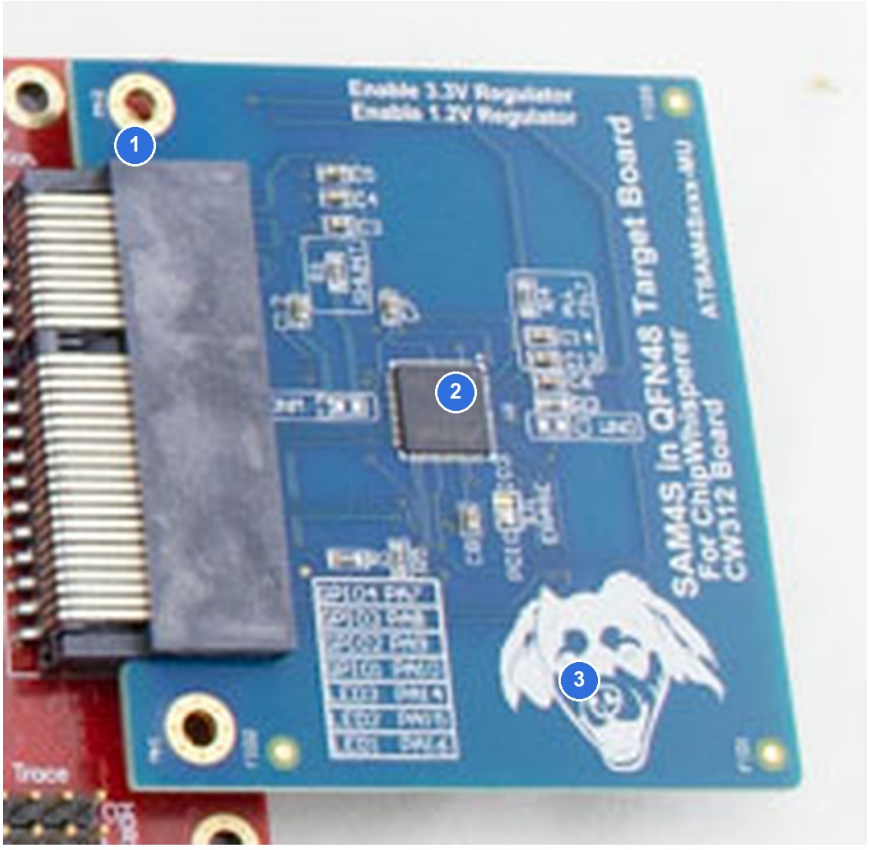



**Which target board? (the kit has two)**

- **X** Green board, text 'ICE40UP5K WLCSP Target': the FPGA target. NOT used in SCA101.
- **OK** Blue board, text 'SAM4S in QFN48 Target Board' (bear logo, ATSAM4Sxxx-MU): this is the one SCA101 uses. Check the printed text, not just the colour.

*Photo: NewAE's Husky starter-kit image (docs/source/Starter Kits). If Python raises 'OSError: timeout' while programming, the wrong board is the first thing to check.*

The two target boards in the kit. Check the printed text, not just the colour.




### Seating the SAM4S board

1. The gold card-edge fingers slide fully into the CW313's J1 socket. The fingers form two unequal groups with a gap, so the board only goes in one way round - do not force it.
2. The ATSAM4S chip (black square), the bear logo and the printed text are on the top face, the side that faces you. In this photo the text runs sideways - that is normal for a board plugged into the CW313.
3. Push until the board sits flat and flush against the socket, with no gold fingers showing.

*Photo: NewAE's jupyter/img/cwhusky.jpg. The text 'Enable 3.3V / 1.2V Regulator' printed on this board applies to the older CW308 UFO board; NewAE's CW313 instructions only list JP1, JP2 and JP4.*

The SAM4S board pushed fully into the CW313's J1 socket.

**YOU SHOULD SEE**

The board sits flat and flush against the socket with no gold fingers showing.

### Step 2 - Husky: which port is which?

The Husky has **two connectors that look like a 20-pin header**. The ribbon cable goes to the **edge** connector next to the USB-C port, labelled **ChipWhisperer 20-Pin Connector**. The black header on the **face** of the Husky (pins D0-D7, CK, DIO, with two gold SMB sockets) is the *User IO* header for the FPGA - **not** where the ribbon goes.




**ChipWhisperer-Husky: what each port is**

- A 'Pos' (yellow, under 'Measurement'): power-measurement input. Cable to a CW313 SMA.
- B 'Neg' (yellow): the second measurement input. Nothing is plugged into it in NewAE's photo. For single-ended use the Husky docs recommend fitting the provided short-circuit cap here (quieter trace).
- C 'Crowbar' (red): glitch output, for the voltage-glitching labs only.
- D ChipWhisperer 20-pin connector (on the end edge, beside USB-C): the ribbon cable to the CW313 goes here.
- E USB-C port: cable to your computer.
- X NOT the target connector: the black 20-pin header on the face (pins D0-D7, CK, DIO, with the two gold SMB sockets) is the User IO header for the FPGA. Do not plug the ribbon here.

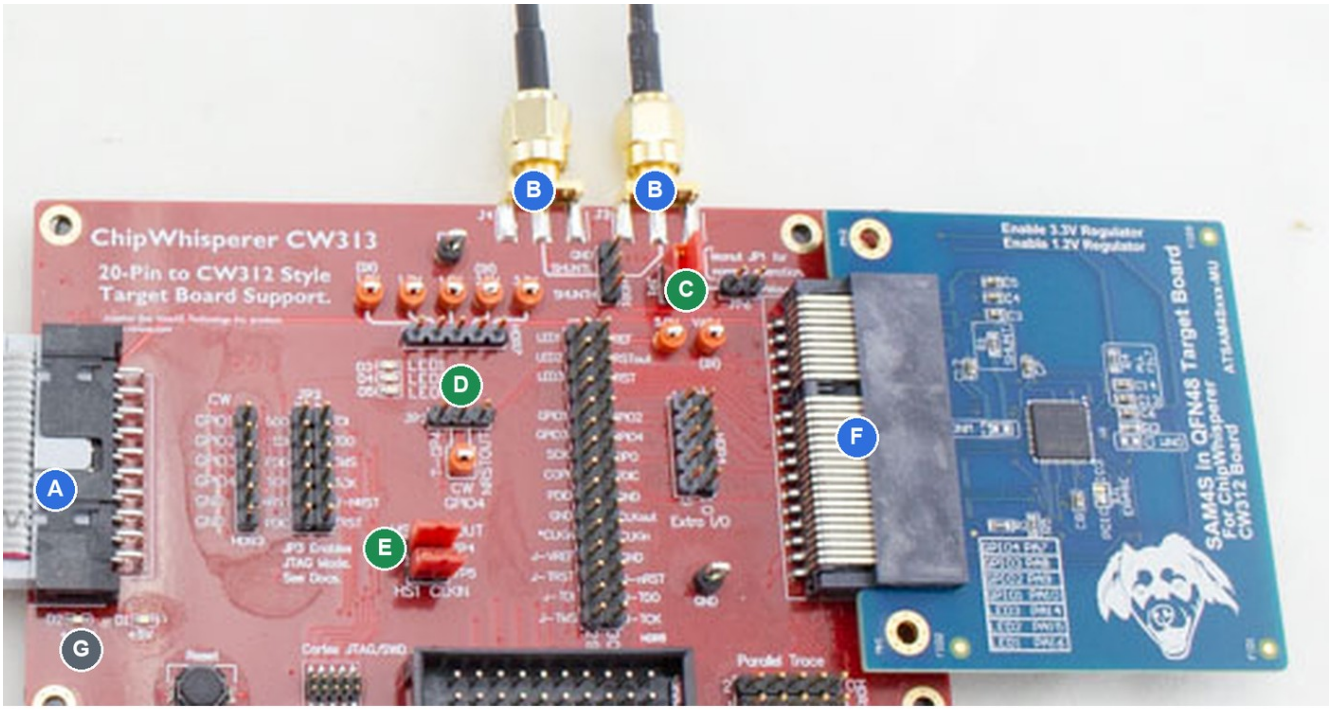
*Source: NewAE's photo jupyter/img/cwhusky.jpg, rotated so the label reads upright; port names from the Husky label and docs/source/Capture/ChipWhisperer-Husky.md.*

The Husky's ports. The red cross marks the User IO header, which is NOT the target connector.

**DO THIS**

(a) Plug the **ribbon** into the Husky's edge connector (D). (b) Plug the **SMA cable** into the Husky's `Pos` socket (A). (c) Leave `Neg` empty - NewAE recommends fitting the short-circuit cap that ships with the Husky there for single-ended measurements. (d) Do **not** plug the USB-C cable in yet.

## Step 3 - CW313: sockets and jumpers




### CW313 board: sockets and jumpers (as shipped)

- **A** J2, 20-pin socket (left): the ribbon cable from the Husky plugs in here.
- **B** J4 and J3, the two SMA sockets (top). They are wired together, so it does not matter which cable goes to which.
- **C** JP1: shunt (jumper cap) fitted - NewAE says JP1 must be connected.
- **D** JP2 chooses what feeds the Husky's GPIO4 trigger input. It must be on T-GPIO4 (the target's GPIO4), which NewAE says is the default. In NewAE's photo no loose jumper cap is visible on JP2; if yours has one, it goes on the T-GP4 side, not the NRSTOUT side. Please check your board.
- **E** JP4 (HS2 CLKOUT) and JP5 (HS1 CLKIN): shunts fitted in NewAE's photo. NewAE's notebook says JP4 must be connected.
- **F** J1, the card-edge socket (right): the SAM4S target board pushes into it.
- **G** Power LEDs (bottom left, printed D2 +3.3V and D1 +5V). D2 shows the 3.3V that the Husky supplies through the ribbon; the Husky can switch that supply on and off from Python (scope.io.target_pwr), so it may be dark until the lab has connected to the Husky. D1 (+5V) is expected to stay off with a Husky: the 20-pin connector only carries 5V on the CW-Pro.

*Source: NewAE's photo jupyter/img/cwhusky.jpg; settings from '1 - Connecting to Hardware.ipynb' (JP2 on T-GPIO4, JP1 and JP4 connected) and docs/source/Targets/CW313.md. Positions are read from the photo - compare with your own board.*

The CW313. Positions are read from NewAE's photo - compare them with your own board.

**DO THIS**

(a) Plug the other end of the **ribbon** into **J2** (A), the black socket on the left. (b) Plug the other end of the **SMA cable** into either SMA socket (B) - the two are wired together. (c) Check the jumpers (C, D, E): **JP1** and **JP4** are connected (a jumper cap is fitted); **JP2** is set to **T-GPIO4**, the default. Leave them as shipped. (d) If you also have the optional **Crowbar** cable (for the voltage-glitching labs), connect Husky `Crowbar` to the *other* SMA socket.

**WATCH OUT**

In NewAE's photo no loose jumper cap is visible on **JP2**. NewAE's notebook says JP2 must be on T-GPIO4 (the default). If yours has a cap on it, it goes on the T-GP4 side of the printed label, not the NRSTOUT side. **Please check your own board.**

### Step 4 - Plug in the USB cable and check

Last, plug the **USB-C** cable into the Husky's USB-C port and into your computer. Use a data-capable cable. Then check:

| | Check | If it is wrong |
|---|---|---|
| [ ] | The target board's printed text says **SAM4S in QFN48 Target Board**, and it is fully seated. | H-1 |
| [ ] | The ribbon is in the Husky's **edge** connector (not the User IO header on the face), and fully seated at both ends. | H-4 |
| [ ] | The `Pos` SMA cable is finger-tight at both ends. `Neg` is empty (or has the short-circuit cap). | H-2 |
| [ ] | `JP2` is on T-GPIO4; `JP1` and `JP4` are connected. | L-44 |
| [ ] | `lsusb \| grep -i 2b3e` prints a line (a Husky shows as `2b3e:ace5`). | L-16, L-17 |

**GOOD TO KNOW**

The CW313 has two power LEDs at its bottom-left corner (printed D2 `+3.3V` and D1 `+5V`). D2 shows the 3.3V the Husky supplies through the ribbon; the Husky can switch that supply on and off from Python, so it may be dark until the lab has connected to the Husky. D1 (+5V) is expected to stay off with a Husky: the 20-pin connector only carries 5V on the CW-Pro.

## 4.3 Section by section

| Section of the notebook | What you do | What you should see |
|---|---|---|
| Introduction and lab layout | Read. Learn the MAIN / SIMULATED / HARDWARE layout. | Nothing to run. |
| Jupyter Background | If you have not used Jupyter before, work through `0 - Introduction to Jupyter Notebooks` first (link in the notebook). | You can run cells with Shift+Enter. |
| Hardware Setup (Husky kit) | Connect the kit as in 4.2. **Skip this if you are only doing the SIMULATED labs.** Ignore the 'Windows driver' note: on Linux the udev rules from step 1.4 do that job. | The kit is wired as in the pictures. |
| Software check 1 - ChipWhisperer imports | Run the cell. | `ChipWhisperer 6.0.0` (the number may differ). |
| Software check 2 - plotting stack | Run the cell. | `numpy 1.26.4 \| holoviews 1.23.2` (holoviews may differ). |
| Software check 3 - build tools | Run the cell. It runs `make` and the Arm compiler from inside Jupyter. | `GNU Make 4.3 ... Built for x86_64-pc-linux-gnu` and then a compiler version like `10.3.1`. |
| Hardware connection test | Run it **only with the Husky plugged in**. It connects, prints details, then disconnects so other notebooks can use the device. | With hardware: `connected: OpenADC`, lines for `fw_version`, `sam_build_date` and `sn`, then `hardware OK (disconnected again)`. |
| Conclusion | Read, then move on to `1 - Connecting to Hardware` and Lab 2_1A. | - |

> **YOU SHOULD SEE (WITHOUT HARDWARE)**
>
> **No hardware connected?** The last check prints `no hardware this run: OSError - Could not find ChipWhisperer. Is it connected?` That is the correct result without a ChipWhisperer - it is a message, not a crash, and every other check should still pass.

## 4.4 What you should be able to do at the end of Lab 0

Lab 0 is finished when every row is true:

| | Outcome | Evidence |
|---|---|---|
| [ ] | ChipWhisperer is installed in your environment and Python can import it. | Check 1 prints a version. |
| [ ] | The plotting and analysis stack works, so the labs can draw traces. | Check 2 prints numpy and holoviews versions. |
| [ ] | You can build firmware from inside Jupyter. | Check 3 prints Make and compiler versions. |
| [ ] | **With a Husky:** Python can open it, read its version and release it, **without** `sudo`. | The hardware test ends with `hardware OK (disconnected again)`. |

| | Outcome | Evidence |
|---|---|---|
| [ ] | You understand the MAIN / SIMULATED / HARDWARE layout and the `VERSION` switch. | You can say what each file does. |
| [ ] | You know that **only one program can hold the Husky at a time.** | You close or restart other notebooks before running a lab. |

**WHAT PASSING MEANS**

**What passing unlocks.** Checks 1 to 3 passed: every `SIMULATED` lab works. The hardware test also passed: the `HARDWARE` labs work too. Next: skim `1 - Connecting to Hardware`, then go on to **Lab 2_1A** (Part 5).

**IF IT DOESN'T**

A check fails? Do not carry on - later labs will fail in confusing ways. Find the message in Part 6.

# Part 5 - Lab 2_1A: Instruction Power Differences

**Lab 2_1A** is the first real use of everything you have installed. A small program runs on the target chip between two **trigger** signals while the ChipWhisperer records the chip's power draw - a **power trace**. You run four different pieces of code (nothing, 20 multiplies, a loop of 20 multiplies, a loop of 20 divides), compare the four traces, and see that **different instructions look different in the power trace**.

There are two notebooks for it, both in `jupyter/courses/sca101/`:

| Notebook | What it is | Open it? |
|---|---|---|
| `Lab 2_1A - Instruction Power Differences (MAIN).ipynb` | The lab: capture (or replay) four traces and compare them. | **Yes, first** |
| `Lab 2_1A - Instruction Power Explorer.ipynb` | The hands-on companion: edit numbers, watch the trace change, measure the clock, count loop trips. It has its own step-by-step PDF, *Lab 2_1A Explorer - Student Guide*. | **Yes, second** |
| `... (HARDWARE).ipynb`, `... (SIMULATED).ipynb` | Back-end helpers that the notebooks above run for you. | **No** |

## 5.1 Open Lab 2_1A (MAIN) and choose the version

Open it the same way as Lab 0 (Jupyter file list: **courses**, then **sca101**). The first code cell holds the settings:

```
SCOPETYPE = 'OPENADC'
PLATFORM  = 'CWHUSKY'
CRYPTO_TARGET = 'NONE'
SS_VER = 'SS_VER_2_1'

VERSION = 'SIMULATED'
```

| Setting | Value for the Husky kit | Meaning |
|---|---|---|
| `SCOPETYPE` | `'OPENADC'` | The kind of capture board (CW-Lite, CW-Pro and CW-Husky all use `'OPENADC'`). |
| `PLATFORM` | `'CWHUSKY'` | The target: the SAM4S microcontroller board in the Husky kit. |
| `CRYPTO_TARGET`, `SS_VER` | `'NONE'`, `'SS_VER_2_1'` | Firmware build options (no crypto library; SimpleSerial version 2.1). |
| `VERSION` | `'SIMULATED'` or `'HARDWARE'` | `'SIMULATED'` replays four recorded traces (no hardware). `'HARDWARE'` uses your Husky. |

**WATCH OUT**

**Check `VERSION` before you run anything.** The course version ships as `'SIMULATED'`, but a copy that someone has already used may say `'HARDWARE'`. Without a Husky connected, `'HARDWARE'` fails at the first cell.

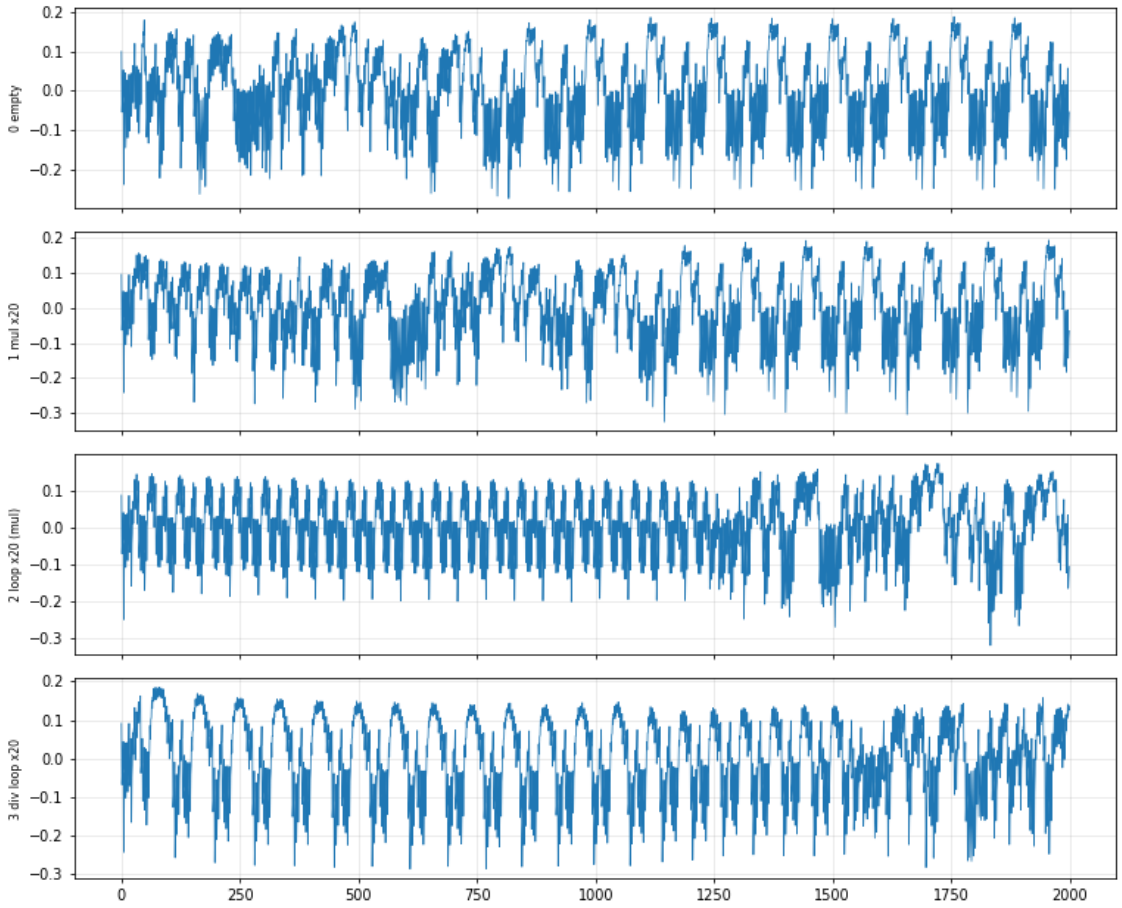
## 5.2 Run it with recorded traces (SIMULATED)

With `VERSION = 'SIMULATED'`, run the cells from the top (**Shift+Enter**, or **Kernel > Restart & Run All**).

**YOU SHOULD SEE**

The setup cells print a check mark and `Simulated capture_trace() ready.`, then `trace length: 2000` and a check mark with `OK to continue!`. Every later `cw.plot(...)` cell draws an interactive plot (zoom and pan tools) and there are no red error boxes. On Ubuntu 22.04 the whole notebook ran top to bottom with no errors.

| Section | Code between the triggers | What you should see |
|---|---|---|
| Empty trace | Nothing (the baseline). | Not much happening: the target idling. |
| Simple instructions | 20 multiplies, one after another. | Hardly any difference from the baseline. Overlay the two plots to compare. |
| Simple instruction loop | A loop that does the multiply 20 times. | Clearly different: a short pattern that repeats. |
| Expensive instructions | A loop that does a divide 20 times. | The same loop pattern, but each trip takes longer. Remember the trace is **upside down** (bigger negative swings = more power). |




The four recorded traces, one per section (drawn here with matplotlib; the notebook's own plots are interactive).

## 5.3 Optional: run it on the real chip (HARDWARE)

Only do this once the kit is connected as in 4.2 and Lab 0's hardware test ended with `hardware OK (disconnected again)`. Then set `VERSION = 'HARDWARE'` and choose **Kernel > Restart & Run All**. The HARDWARE helper notebook does the following for you:

1. Makes a private copy of the firmware, `firmware/mcu/simpleserial-base-lab2`, so the original is never touched.
2. Connects to the ChipWhisperer, runs `scope.default_setup()` and power-cycles the target.
3. Builds and flashes a baseline program, so the first capture works immediately.
4. For each section, writes different C code between the trigger calls, runs `make`, flashes the chip and captures a trace (about 10 to 20 seconds each).

> **YOU SHOULD SEE**
>
> The setup prints `INFO: Found ChipWhisperer`, the path of the working copy, and finally `Hardware ready - baseline simpleserial-base flashed.` Each section then rebuilds and reflashes before plotting.

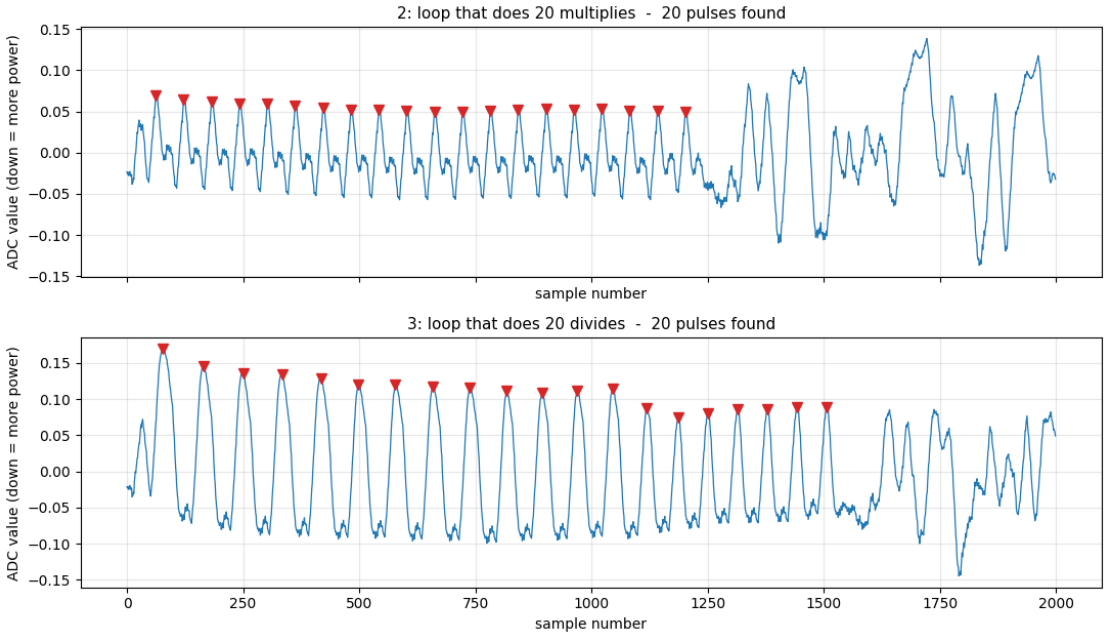
**GOOD TO KNOW**

**What has been checked on Linux.** All four code variants (empty, 20 multiplies, the multiply loop and the divide loop) build cleanly with the Arm compiler on Ubuntu 22.04. On the Ubuntu 24.04 VMware VM the Husky connects, the SAM4S is programmed and the serial link answers. A complete HARDWARE run of Lab 2_1A on the Linux VM has **not** been recorded yet, so report anything unexpected using the block in 6.8.

## 5.4 Do the Explorer notebook

Now open `Lab 2_1A - Instruction Power Explorer.ipynb`. Where the MAIN lab shows you four traces, the Explorer lets you **change the numbers and watch the trace change**: zoom and smooth a trace, measure the clock, count loop trips, see why the divide loop speeds up, and predict what a different loop count would look like. Steps 1 to 6 need no hardware; Step 7 edits the C code on the real chip. Follow the separate PDF **Lab 2_1A Explorer - Student Guide** step by step.

| Explorer step | You measure or see | The number to look for |
|---|---|---|
| 2 | Samples per clock cycle | **4** |
| 4 | Pulses in each loop | **20** (multiply loop: 60 samples = 15 cycles per trip) |
| 5 | Divide loop trip length | Starts at 22 cycles and shrinks to 16 as the number gets smaller (correlation r = +0.97) |
| 6 | Model of any loop count | `prediction: 20 trips x 60 samples = 1200 samples` |




Explorer Step 4: the pulse finder marks 20 trips in both loops (the divide loop's pulses move closer together).

**YOU SHOULD SEE**

On Ubuntu 22.04 the Explorer ran top to bottom in a real Jupyter Notebook 7 session with no errors, and the two slider widgets in Step 6 redrew the plot as they were dragged.

## 5.5 What you should be able to do at the end of Lab 2_1A

| | Outcome | Evidence |
|---|---|---|
| [ ] | You can plot four power traces and say which section is quietest. | The Empty trace plot, compared with the others. |
| [ ] | You can recognise a loop in a trace: the same pulse repeating. | The loop sections show a repeating pattern. |
| [ ] | You know a divide takes longer than a multiply, and that the trace is upside down. | Each divide trip is wider; larger negative swings mean more power. |
| [ ] | Explorer: you measured 4 samples per clock cycle and counted 20 pulses in each loop. | Explorer Steps 2 and 4. |
| [ ] | Explorer: you can explain why the divide loop speeds up (the data changes the timing). | Explorer Step 5 and Step 8, question 3. |
| [ ] | **With a Husky:** you changed the C code, rebuilt, flashed and captured your own trace. | Explorer Step 7, or the MAIN notebook in HARDWARE mode. |

**WHAT PASSING UNLOCKS**

**Next:** Lab 2_1B uses the same idea to break a password check - the chip works a little longer for each correct character, and the difference shows up in the trace. It has its own Explorer notebook, `Lab 2_1B - Password Bypass Explorer.ipynb`, with a step-by-step PDF. Make sure `cwtraces` is installed first (step 2.2).

# Part 6 - Troubleshooting

Find the message you see, read what it means, try the fix. Entries are grouped by where in the process things go wrong. The small **Source** line says how the entry was established:

| Source tag | Meaning |
|---|---|
| **Reproduced** | We ran it on Ubuntu (22.04 in WSL2, 24.04 in a VMware VM) while writing this guide and saw this result. |
| **NewAE docs** | Taken from NewAE's ChipWhisperer documentation. |
| **Source code** | Taken from the ChipWhisperer or notebook source code. |
| **Husky kit bring-up** | Seen while bringing up a real Husky + CW313 + SAM4S kit. |
| **User report** | Reported by someone following this guide on their own computer. |
| **General Linux** | Standard Linux behaviour; not specific to ChipWhisperer. |

## 6.1 Installing packages (apt, venv, pip)

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-1 | `E: Unable to locate package gcc-arm-none-eabi` (or any other package) | The package list is empty or stale, which is normal on a fresh VM.<br>Source: General Linux | `sudo apt update`, then repeat the install. |
| L-2 | `The virtual environment was not created successfully because ensurepip is not available` | `python3-venv` is not installed.<br>Source: General Linux | `sudo apt install python3-venv` (install the exact package the message names if it differs), then `rm -rf ~/.cwvenv` and repeat step 1.3. |
| L-3 | `error: externally-managed-environment` | You ran `pip` outside the virtual environment. Debian 12+ and Ubuntu 23.04+ block this on purpose.<br>Source: General Linux | `source ~/.cwvenv/bin/activate`, check the prompt shows `(.cwvenv)`, repeat. Do **not** use `--break-system-packages`. |
| L-4 | `pip install -e .` stops with `Failed building wheel for numpy` (or a long error about numpy), and `python --version` says 3.13 or higher | ChipWhisperer pins `numpy<=1.26.4`. numpy 1.26.4 has **no pre-built package for Python 3.13+** (pip's index offers pre-built numpy for 3.13 only from 2.1 upward), so pip tries to compile an old numpy and fails.<br>Source: Reproduced (the missing-wheel check); the build failure is the expected result | Use Python 3.9 to 3.12. On Ubuntu, one way is the deadsnakes PPA: `sudo add-apt-repository ppa:deadsnakes/ppa`, `sudo apt install python3.12 python3.12-venv python3.12-dev`, then `rm -rf ~/.cwvenv` and `python3.12 -m venv ~/.cwvenv`. Other distributions: `pyenv` or `uv python install 3.12`. (Not verified in this guide.) |

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-5 | `UserWarning: compilation failed. Installing pure python package` during `pip install` | `fastdtw` could not compile its fast C++ extension (no C++ compiler installed) and installed the slower pure-Python version instead. pip still reports success. Source: Reproduced | Harmless; ignore it. To get the fast version: `sudo apt install build-essential`, then `python -m pip install --force-reinstall --no-cache-dir --no-deps fastdtw==0.3.4`. |
| L-6 | `usermod: group 'chipwhisperer' does not exist` | The `groupadd` line in step 1.4 was skipped or failed. Source: General Linux | `sudo groupadd -fr chipwhisperer`, then repeat the `usermod -aG chipwhisperer $USER` line. |
| L-7 | `usermod: group 'plugdev' does not exist` | Your distribution does not have that group. Source: General Linux | Ignore it. The rules use the `chipwhisperer` group, not `plugdev`. |
| L-8 | The `jupyter` folder is empty, missing, or `git clone` says `destination path 'jupyter' already exists and is not an empty directory` | The clone was not run from inside `~/chipwhisperer`, or `jupyter` already holds files from an earlier attempt. Source: General Linux | `cd ~/chipwhisperer`. If `jupyter` is empty or missing, run the clone command from step 2.1. If it holds an old copy, `mv jupyter jupyter.old` and clone again. |
| L-9 | `Could not resolve host: github.com`, `unable to access`, or pip `Read timed out` | No network, or a slow/blocked connection. Source: General Linux | `ping -c 1 github.com`. On a VM check the network adapter (NAT or bridged). Behind a proxy set `https_proxy`. For slow links add `--default-timeout=100` to pip. |
| L-10 | `Defaulting to user installation because normal site-packages is not writeable` when you run `pip` | You are using the **system** pip, not the environment's. Packages go to `~/.local`, not into `.cwvenv`, and later `import` errors follow. Source: Reproduced | Activate the environment (`source ~/.cwvenv/bin/activate`) and always type `python -m pip ...`, which uses the active Python's own pip. Then repeat the install. |
| L-34 | `firmware (1.5.0) is outdated - latest is 1.7.0` when you connect, or `7.0.0` after the install | You are on NewAE's current `develop` (7.0.0), which bundles Husky firmware 1.7.0 and needs Python 3.11+ and numpy 2. The labs were validated on the 6.0.0 software, and your Husky is on firmware 1.5.0. Source: Husky kit bring-up | Switch to the pinned version: `cd ~/chipwhisperer`, `git checkout 7b3206b0`, then `python -m pip install -e .` (recreate the environment first if pip complains - see L-35). Check that `cw.__version__` prints `6.0.0`. Do **not** update the Husky's firmware just to silence the message. |
| L-35 | After switching versions, pip warns that scipy, zarr, numcodecs or contourpy `requires numpy>=2` | Packages installed for 7.0.0 are still in the environment, but the 6.0.0 software pins an older numpy. Source: Husky kit bring-up | Start clean: `deactivate`, `rm -rf ~/.cwvenv`, then repeat steps 1.3, 1.5 and 2.2. |

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-48 | `E: Unable to locate package libnewlib-arm-none-eabi` and `E: Unable to locate package python3` - note the **two spaces** after `package`, although `sudo apt update` was run | The `apt install` command was pasted from a PDF as one line that still contained `\` (a backslash and a space). The shell turns each `\` into a literal space inside the next word, so apt looks for packages called " libnewlib-arm-none-eabi" and " python3" - names that do not exist. The mechanism was reproduced with a simulated install on Ubuntu 22.04. Source: User report (Ubuntu desktop); mechanism reproduced | Use the **single-line** commands in step 1.1 and paste **one line at a time** (or type them). If you ever have to run a multi-line command from a document, type it, or save it into a file and run `bash thefile`. This guide's earlier version split the `apt install` command over three lines with backslashes; version 1.3 fixes that. |
| L-50 | `cp: cannot stat '50-newae.rules': No such file or directory` (step 1.4); or `ERROR: ... does not appear to be a Python project: neither 'setup.py' nor 'pyproject.toml' found` (step 1.5); or `fatal: not a git repository` (a `git` command in step 1.2) | You are in the wrong folder. These commands use files that sit **inside** the `chipwhisperer` folder, but the prompt ends in `~` (your home folder), or in some other folder. A new terminal window always starts in `~`. Source: User report (Ubuntu 24.04 desktop, photo of the failing `cp`) | Run `cd ~/chipwhisperer`, then `ls`: it must list `pyproject.toml`, `50-newae.rules`, `firmware` and `software`. Then repeat the command that failed. If `cd ~/chipwhisperer` says `No such file or directory`, step 1.2 never ran: go back and run its four commands (clone, `cd`, checkout). The virtual environment does **not** need to be recreated - it lives in `~/.cwvenv`, not in the ChipWhisperer folder; just make sure the prompt starts with `(.cwvenv)`. |
| L-49 | The check after step 2.2 prints `5.6.1` (anything but `6.0.0`) or a numpy starting with `2`; or Lab 2_1B fails with `TypeError: Chart.__init__() missing 1 required positional argument: 'data'`; or pip printed `ERROR: ... requires numpy>=2` | pip replaced ChipWhisperer 6.0.0 with the old 5.6.1 and installed numpy 2. ChipWhisperer 6.0.0 needs numpy 1.26.4 or older, but the newest `scipy` (1.18) needs numpy 2 or newer; `requirements.txt` asks for `chipwhisperer` without a version, so pip resolved the conflict by dropping your editable copy. In 5.6.1, `cw.plot()` called without arguments does not exist, which is what breaks Lab 2_1B. Source: Reproduced (fresh Python 3.12 install on Ubuntu) | Start clean: `deactivate`, `rm -rf ~/.cwvenv`, then repeat steps 1.3, 1.5 and 2.2 - using the 2.2 command **with** `"numpy<=1.26.4"`. The check must print `6.0.0 1.26.4`. With the limit, the same clean install ran Lab 0, Lab 2_1A, both Explorers, Lab 2_1B and the solved notebook with no errors. (Python 3.10 does not trigger this, because its newest scipy still accepts numpy 1.x.) |

## 6.2 Jupyter and notebook kernels

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-11 | `jupyter: command not found` | The environment is not active, or step 2.2 did not finish.<br>Source: General Linux | `source ~/.cwvenv/bin/activate`. `which jupyter` must print a path inside `.cwvenv`. If not, repeat step 2.2. |
| L-12 | `ModuleNotFoundError: No module named 'chipwhisperer'` (or numpy, holoviews...) inside a notebook | The notebook's kernel is a different Python from your environment.<br>Source: General Linux | In a cell run `import sys; print(sys.executable)`; it must contain `.cwvenv`. If not, stop Jupyter (`Ctrl+C`), activate the environment, start again from that same terminal. Still wrong: `python -m ipykernel install --user --name cwvenv --display-name "ChipWhisperer (cwvenv)"`, then **Kernel > Change Kernel** in the notebook. |
| L-13 | No browser opens, or you are working over SSH / on a headless VM | Jupyter cannot start a browser there.<br>Source: General Linux | Start with `jupyter notebook --no-browser`, copy the `http://localhost:8888/...token=...` address into a browser. From another computer: `ssh -L 8888:localhost:8888 <user>@<vm-address>`, then browse to `http://localhost:8888` on your own computer. |
| L-14 | `The port 8888 is already in use, trying another port.` | An older Jupyter is still running.<br>Source: General Linux | Use the address it prints (the port may be 8889 or higher), or stop the old one with `jupyter notebook list` to find it and `Ctrl+C` in its terminal. |
| L-15 | The browser shows a Jupyter login page asking for a `Password or token` | You opened `localhost:8888` by hand instead of using the address Jupyter printed, which contains the token.<br>Source: General Linux | Copy the full `http://localhost:8888/...?token=...` address from the terminal where Jupyter started, or run `jupyter notebook list` (environment active) to print it again. |
| L-36 | `Javascript Error: IPython is not defined` after a cell with `%matplotlib notebook` (for example in `ChipWhisperer Setup Test`) | An old matplotlib magic that Jupyter Notebook 7 does not support. It only affects that one plot.<br>Source: Husky kit bring-up | Harmless; ignore it. The scope test further down the notebook is unaffected. |

## 6.3 USB devices and permissions

Work through this section in order. It is the most common source of trouble on Linux.

```
lsusb | grep -i 2b3e                    # 1. does Linux see the device at all?
ls -l /etc/udev/rules.d/50-newae.rules # 2. are the rules installed?
id                                      # 3. is 'chipwhisperer' in your groups?
```

Then look at the device node itself. `lsusb` prints `Bus 001 Device 004` (your numbers will differ); the matching node is `/dev/bus/usb/001/004`:

```
ls -l /dev/bus/usb/001/004
```

**YOU SHOULD SEE**

A working setup prints `crw-rw-r-- 1 root chipwhisperer ...`. The group column must say **chipwhisperer**. If it says `root`, the rules did not apply to this plug-in: reload and unplug/replug (below).

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-16 | `OSError: Could not find ChipWhisperer. Is it connected?` | Python sees no NewAE USB device at all. Source: Reproduced | Run `lsusb \| grep -i 2b3e`. **Nothing printed:** Linux cannot see it - re-seat the USB-C cable (use a data cable, not a charge-only one), try another port, and in a virtual machine fix USB pass-through (6.5). **A line printed:** the device is there but Python cannot use it - see L-17. |
| L-17 | `Could not open USB device`, `LIBUSB_ERROR_ACCESS`, or `Unable to communicate with found ChipWhisperer. Check that another process isn't connected to it and that you have permission to communicate with it.` | You lack permission on the USB device node, or another program already has the device. The ChipWhisperer code prints exactly these two hints. Source: Source code | 1) Rules installed? `ls /etc/udev/rules.d/50-newae.rules`. 2) `sudo udevadm control --reload-rules` then **unplug and replug** the device. 3) Device node group is `chipwhisperer` (see above). 4) `id` lists `chipwhisperer`; if not, log out and in (L-19). 5) Rule out another process (L-22). |
| L-18 | It works with `sudo` but not without | The same permission problem as L-17. Source: General Linux | Fix the permissions rather than using `sudo`. **Never run Jupyter with** `sudo`: it works once, then leaves root-owned files in your folders. |
| L-19 | `id` does not list `chipwhisperer` even though you ran `usermod` | Group membership is read at login. Source: General Linux | Log out and back in, or reboot. `newgrp chipwhisperer` fixes one terminal only (you must then activate the environment and start Jupyter from that terminal). |
| L-20 | `Permission denied: '/dev/ttyACM0'` (or `cw_serial0`) | You are not in the `dialout` group, which owns serial ports on many distributions. Source: NewAE docs | `sudo usermod -aG dialout $USER`, then log out and in again. |

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-21 | Serial port is busy, returns junk, or is flaky on Ubuntu | ModemManager probes every `/dev/ttyACM*` port with modem commands and interferes with the ChipWhisperer's serial port. Source: NewAE docs | Quick test: `sudo systemctl stop ModemManager`, replug. Permanent: `sudo systemctl disable --now ModemManager`. NewAE's alternative (ModemManager 1.12+) is to append `, ENV{ID_MM_DEVICE_IGNORE}="1"` to the end of each `SUBSYSTEM=="tty"` line in `/etc/udev/rules.d/50-newae.rules`, then reload the rules and replug. (On Ubuntu 24.04 ModemManager was running but ignored the Husky - `mmcli -L` printed nothing - so no rule was needed. Test before changing anything.) |
| L-22 | Connection fails after it worked once; or `Unable to communicate...` and permissions look fine | Another program still holds the device: a second notebook, an old kernel, or a leftover Python process. Only one program can talk to a ChipWhisperer at a time. Source: Husky kit bring-up | In Jupyter: **Kernel > Shut Down All Kernels** (and close other lab tabs). In a terminal: `ps aux \| grep -i ipykernel` and stop leftovers with `kill <pid>`. Unplug and replug the Husky. Restart the kernel before re-running the notebook. Do not run notebooks on another computer or in a VM at the same time. |
| L-37 | `usb1.USBErrorTimeout` on the very first FPGA upload when you connect (`cw.scope()` fails), even though permissions and pass-through look right | The Husky is **stuck** - it needs a power cycle. This is not caused by Linux, the VM, USB 2.0/3.0 or the ChipWhisperer version: the same failure happened on Windows and was cleared the same way. Source: Reproduced | **Physically unplug and replug the Husky**, and everything attached to its 20-pin connector (power-cycle the whole setup), then connect again. Try this before blaming the VM or the udev rules. |
| L-38 | `dmesg` shows `usbfs: process ... did not claim interface 0 before use` | A harmless kernel notice from the first USB access. Source: Reproduced | Ignore it. A healthy Husky in a VM shows up on the high-speed (480M) controller with `cdc_acm` bound to its serial port. |
| L-23 | `Could not find bootloader serial port` (only when updating ChipWhisperer firmware) | The device in bootloader mode is not visible to this machine. Source: NewAE docs | Make sure it is plugged in (and passed through to the VM). After the rules are applied, a `/dev/cw_bootloader*` device should appear: `ls /dev/cw_bootloader*`. |

## 6.4 Compilers and firmware builds

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-24 | `make: arm-none-eabi-gcc: No such file or directory` or `arm-none-eabi-gcc: command not found` | The Arm compiler is not installed. (Lab 0 check 3 prints its version when it is: 10.3.1 on Ubuntu 22.04.) Source: General Linux (the working version output was reproduced) | `sudo apt install gcc-arm-none-eabi libnewlib-arm-none-eabi`, then `arm-none-eabi-gcc -dumpversion`. |
| L-25 | Compiler stops with `fatal error: stdint.h: No such file or directory`, `cannot find -lc_nano`, or other missing C library files | The newlib C library for the Arm compiler is missing. apt lists it only as a *recommended* package of `gcc-arm-none-eabi`, so it is skipped when recommends are turned off. Source: General Linux (the apt dependency was checked on Ubuntu 22.04) | `sudo apt install libnewlib-arm-none-eabi`, then build again. |
| L-26 | `No such file or directory: '../../../firmware/mcu/basic-passwdcheck'` in a HARDWARE notebook | The notebooks are not inside the `chipwhisperer` folder (for example, `chipwhisperer-jupyter` was cloned on its own). Source: Source code | Check `ls ~/chipwhisperer/firmware/mcu/basic-passwdcheck` exists and the notebooks are in `~/chipwhisperer/jupyter/courses/sca101/`. If not, redo step 2.1. |
| L-27 | `make` prints a very long list of platforms and ends with `Stop.` | `make` was run by hand without `PLATFORM=`. Source: General Linux | Use the full command below. The notebooks pass these settings for you. |
| L-28 | `avr-gcc: command not found` (XMEGA / CW-Lite labs only) | The AVR compiler is missing. The SAM4S in the Husky kit does not need it (and NewAE's live install page no longer lists `gcc-avr` and `avr-libc`). Source: General Linux | `sudo apt install gcc-avr avr-libc` - only if you have an AVR/XMEGA kit. |

### A manual build, for testing the toolchain outside Jupyter (this is the Husky + SAM4S build used by the labs):

```
cd ~/chipwhisperer/firmware/mcu/basic-passwdcheck
make PLATFORM=CWHUSKY CRYPTO_TARGET=NONE SS_VER=SS_VER_2_1 -j4
```

**YOU SHOULD SEE**

The output ends with a box saying `Built for platform CW-Husky Microchip SAM4S`, and the folder now contains `basic-passwdcheck-CWHUSKY.hex`. (This exact command was run on Ubuntu 22.04.)

## 6.5 Virtual machines

A virtual machine only sees the USB device if the VM software **passes it through**. NewAE's documentation describes this for VirtualBox; VMware works similarly. The rules below apply after the Linux inside the VM is set up as in Part 1.

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-29 | The ChipWhisperer shows in the host's device list but `lsusb` inside the VM shows nothing | USB pass-through is not set up, or the VM only has a slow USB 1.1 controller. Source: NewAE docs | VirtualBox: install the **Extension Pack** from the VirtualBox download page (NewAE states it is necessary for the VM to interact with ChipWhisperer hardware); **Settings > USB**: choose a USB 2.0 or 3.0 controller; add a **USB filter** for the ChipWhisperer. Or attach it by hand each boot: **Devices > USB > ChipWhisperer**. **VMware Workstation Player:** with the VM running, **Player > Removable Devices > NewAE ChipWhisperer Husky > Connect (Disconnect from Host)**; it disappears from the host while the VM has it (see L-39). |
| L-30 | Pass-through works for the Husky but not after it resets, or while updating firmware | The device re-appears under a different USB ID. The bootloader shows up as `03eb:6124`, not `2b3e:...`, so a filter for the Husky alone does not catch it. Source: Source code | Add a second filter for vendor `03eb` product `6124`, or attach it manually from **Devices > USB** when it appears. Use a filter on **vendor ID** `2b3e` rather than one exact product. |
| L-31 | VirtualBox cannot attach the device on a **Linux host** | Your host user cannot access USB for VirtualBox. Source: NewAE docs | `sudo usermod -aG vboxusers <your-username>` on the host, then log out and in again (or restart). |
| L-32 | The device is seen in the VM sometimes and not other times | The host still has it open (for example a Windows Jupyter, an old Python window, or a driver tool), or a USB filter is fighting with a manual attach. Source: Husky kit bring-up | Close ChipWhisperer programs on the host first. NewAE's advice if filters misbehave: untick the filters in **Settings > USB** and attach manually from **Devices > USB**. |
| L-33 | You cannot reach Jupyter running inside the VM from your own browser | The VM's network is NAT-only. Source: NewAE docs | Forward port 8888 in the VM software, or use SSH: `ssh -L 8888:localhost:8888 <user>@<vm-address>`. Or just use the VM's own browser. |
| L-39 | VMware: the Husky does not show up in the VM after you click Connect | The host still has the device, or another VM or program holds it. Verified setup: Ubuntu 24.04 in VMware Workstation 17 Player. Source: Reproduced | Use **Disconnect from Host** when you connect. Inside the VM `lsusb \| grep -i 2b3e` must print `2b3e:ace5`. Close any ChipWhisperer program on the host first, and use L-37 if the scope still will not connect. |

## 6.6 The Husky kit (any operating system)

| # | What you see | What it means | Fix |
|---|---|---|---|
| H-1 | `OSError: timeout` (in `Samba.chip_id` or `read_word`) when programming the target with `SAM4SProgrammer` | The SAM4S target board is not the one on the CW313 (the look-alike **iCE40 FPGA** board is mounted), or it is not fully seated. Source: Husky kit bring-up | Check the board's printed text says SAM4S in QFN48 (4.2, step 1). Remove, re-seat it flush on the CW313, check the 20-pin ribbon, then unplug and replug the Husky and run the cell again. |
| H-2 | The power trace is almost flat (tiny amplitude) and the attack returns garbage | The **Pos** SMA cable is not connected properly between the Husky and the CW313. Source: Husky kit bring-up | Reseat both ends of the Pos cable (finger-tight) as in 4.2, then recapture. |
| H-3 | `LIBUSB_ERROR_NO_DEVICE` or `SAM3U Serial buffers OVERRUN` partway through a run | A USB hiccup, often a marginal cable or hub. Source: Husky kit bring-up | Use a short data-capable USB-C cable straight into the computer (no hub), then **Kernel > Restart & Run All**. |
| H-4 | The Husky connects but the target never answers or cannot be programmed, and the ribbon is in the Husky's **face** header | The ribbon must go to the Husky's edge connector labelled 'ChipWhisperer 20-Pin Connector', beside USB-C. The header on the face (D0-D7, CK, DIO) is the User IO header. Source: NewAE docs | Move the ribbon to the edge connector (4.2, step 2) and check it is fully seated at both ends. |

## 6.7 Lab 2_1A and the Explorers

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-40 | `FileNotFoundError: ... traces/lab2_1A_trace0.npy` (SIMULATED) | The recorded traces are missing. They were deleted upstream (commit d281164) and restored on the course branch, so NewAE's own copy of the notebooks does not have them. Source: Source code | Check `ls ~/chipwhisperer/jupyter/courses/sca101/traces` shows four `.npy` files. If not, you cloned the wrong notebooks - redo step 2.1. |
| L-41 | `NameError: name 'capture_trace' is not defined`, or `ValueError: VERSION must be 'HARDWARE' or 'SIMULATED'` | The `%run` cell did not run, or `VERSION` is mistyped. Source: Source code | Fix `VERSION` (exactly `'SIMULATED'` or `'HARDWARE'`) and use **Kernel > Restart & Run All** so the cells run in order. |

| # | What you see | What it means | Fix |
|---|---|---|---|
| L-42 | `RuntimeError: firmware build failed` with compiler text above it (HARDWARE) | `make` could not build the firmware: no Arm compiler, no newlib, or the notebooks are not inside `~/chipwhisperer`. Source: Source code | Run Lab 0 check 3, then see L-24, L-25 and L-26. |
| L-43 | `OSError: timeout` or `Verify FAILED` while flashing (HARDWARE) | Wrong or unseated target board, or a stuck Husky. Source: Husky kit bring-up | See H-1 (target board) and L-37 (power-cycle the Husky), then run the Setup cells again. |
| L-44 | `IOError: capture_trace timed out 3x - check trigger (CW313 JP2 = T-GPIO4) and target` | The scope never saw the trigger. Source: Source code | Check CW313 jumper `JP2` is on **T-GPIO4** and the target is powered over the ribbon, then run the Setup cells again. |
| L-45 | `Read timed out:` or zero bytes back from the target after flashing it yourself | The SAM4S firmware runs from the Husky's external clock and only answers after the target has been reset. The lab's setup cell power-cycles the target for you; a script of your own must do it. Source: Reproduced | Re-run the lab's Setup cell. In your own script: `scope.default_setup()`, then `scope.io.nrst = 'low'`, wait 0.25 s, `scope.io.nrst = 'high_z'`. |
| L-46 | Explorer problems: wrong counts, missing sliders, `RuntimeError: Run the Step 2 cell first` | See the troubleshooting section of the Explorer guide. Source: Reproduced | Open *Lab 2_1A Explorer - Student Guide* and look up entries E-1 to E-15. |
| L-47 | `ModuleNotFoundError: No module named 'cwtraces'` (Lab 2_1B and later SIMULATED labs, and the 2_1B Explorer) | The recorded-trace package is not installed. It is not in `jupyter/requirements.txt`, so the standard install steps do not install it. Source: Reproduced | You skipped part of step 2.2. In your environment run `python -m pip install cwtraces "numpy<=1.26.4"` (it installed version 0.2.0 and left numpy at 1.26.4). Then restart the notebook's kernel and run it again. |

### 6.8 Still stuck? Collect this and ask for help

Run this block in a terminal with the environment active and the ChipWhisperer plugged in. It prints everything someone needs to see. Paste the **whole** output into your question along with the exact error text and the step you were on.

```
cat /etc/os-release | head -2
python --version; which python; which jupyter
python -m pip list 2>/dev/null | grep -iE "chipwhisperer|numpy|libusb1|cwtraces"
git -C ~/chipwhisperer log -1 --oneline
id
lsusb | grep -i 2b3e
ls -l /etc/udev/rules.d/50-newae.rules
BUSDEV=$(lsusb -d 2b3e: | head -1 | sed -E 's|Bus ([0-9]+) Device ([0-9]+).*|\1/\2|')
ls -l /dev/bus/usb/$BUSDEV
ls -l /dev/cw_* 2>/dev/null
sudo dmesg | tail -20
```

Where to ask: the NewAE forum at forum.newae.com, or the issue tracker at github.com/newaetech/chipwhisperer/issues. If you are on a course, ask your instructor first.

# You are done when...

Check each line. If every one is true, you are ready for **Lab 2_1B**.

| | Check |
|---|---|
| [ ] | `(.cwvenv)` shows in your prompt, and `python -c "import chipwhisperer as cw, numpy; print(cw.__version__, numpy.__version__)"` prints `6.0.0 1.26.4`. |
| [ ] | `git -C ~/chipwhisperer log -1 --oneline` starts with `7b3206b0`. |
| [ ] | `id` lists `chipwhisperer`, and `/etc/udev/rules.d/50-newae.rules` exists. |
| [ ] | `~/chipwhisperer/jupyter/courses/sca101/` contains the lab notebooks, next to `~/chipwhisperer/firmware`. |
| [ ] | `python -c "import cwtraces"` prints no error. |
| [ ] | `jupyter notebook` starts from the environment, and `sys.executable` in a notebook contains `.cwvenv`. |
| [ ] | Lab 0 software checks 1 to 3 all print a version and no error. |
| [ ] | **If you have a Husky:** the kit is wired as in 4.2 and the hardware test ends with `hardware OK (disconnected again)`. |
| [ ] | Lab 2_1A (MAIN) runs top to bottom in `SIMULATED` mode with no red error boxes. |
| [ ] | The Explorer: you measured 4 samples per clock cycle and found 20 pulses in each loop. |
| [ ] | **If you have a Husky:** Lab 2_1A runs in `HARDWARE` mode, or you changed the C code in Explorer Step 7. |

# Appendix A - Quick reference

The whole install in one place, in order. Run each block from inside the folder named above it. **Paste one line at a time** (see the note at the start of Part 1): each line is a separate command.

### Once, from anywhere

```
sudo apt update && sudo apt upgrade
sudo apt install make git avr-libc gcc-avr gcc-arm-none-eabi libnewlib-arm-none-eabi
sudo apt install libusb-1.0-0-dev usbutils python3 python3-venv python3-dev
cd ~/
git clone https://github.com/newaetech/chipwhisperer
cd chipwhisperer
git checkout 7b3206b0
python3 -m venv ~/.cwvenv
source ~/.cwvenv/bin/activate
```

### Once, inside ~/chipwhisperer

```
cd ~/chipwhisperer
sudo cp 50-newae.rules /etc/udev/rules.d/50-newae.rules
sudo udevadm control --reload-rules
sudo groupadd -fr chipwhisperer
sudo usermod -aG chipwhisperer $USER
sudo usermod -aG plugdev $USER
python -m pip install -e .
git clone -b sca101-lab0-lab2-complete https://github.com/LFB1937/chipwhisperer-jupyter jupyter
python -m pip install -r jupyter/requirements.txt cwtraces "numpy<=1.26.4"
python -c "import chipwhisperer as cw, numpy; print(cw.__version__, numpy.__version__)"
```

The last line must print `6.0.0 1.26.4` (see step 2.2 and L-49).

Then log out and in (or reboot), and unplug / replug the ChipWhisperer.

### Every session

```
source ~/.cwvenv/bin/activate
cd ~/chipwhisperer/jupyter
jupyter notebook
```

### Handy diagnostics

| Command | What it tells you |
|---|---|
| `lsusb \| grep -i 2b3e` | Whether Linux sees a NewAE device. |
| `id` | Which groups your current login has (look for `chipwhisperer`). |
| `which python`, `which jupyter` | Whether you are using the virtual environment. |
| `python -m pip list \| grep -i chipwhisperer` | The installed version and, for an editable install, its folder. |
| `ls /dev/cw_*` | Serial devices created by the rules (`cw_serial*`, `cw_bootloader*`). |
| `sudo dmesg \| tail -20` | Kernel messages about USB devices appearing, disconnecting or failing. |

# Appendix B - Glossary

| Term | Meaning |
|---|---|
| **apt** | Ubuntu/Debian's package installer. `sudo apt install x` installs software `x` system-wide. |
| **virtual environment (venv)** | A private folder holding its own Python and packages. `source ~/.cwvenv/bin/activate` switches it on for the current terminal. |
| **pip** | Python's package installer. Inside an environment it installs only into that environment. |
| **editable install (`-e`)** | Installs a package by linking to its folder instead of copying it. |
| **udev rules** | Files in `/etc/udev/rules.d/` that tell Linux what permissions and names to give a device when it is plugged in. |
| **group** | A named set of users. Files and devices can be limited to a group. Your login only learns about new groups when you log in again. |
| **USB vendor / product ID** | Two numbers identifying a USB device. NewAE's vendor ID is `2b3e`; the Husky is `2b3e:ace5`. |
| **Jupyter kernel** | The Python process that runs a notebook's cells. It must be the one from your virtual environment. |
| **firmware / `.hex`** | The program that runs on the target chip. `make` compiles it to a `.hex` file that is flashed onto the chip. |
| **target / capture board** | The *target* is the chip under test (SAM4S). The *capture board* (Husky) measures its power. |
| **SIMULATED / HARDWARE** | The two lab back-ends: recorded traces, or live measurements from your ChipWhisperer. |
| **detached HEAD** | What git says after `git checkout 7b3206b0`: you are on one fixed version (a commit), not on a branch. Normal here. |
| **power trace / trigger** | A recording of the target's power draw, started and stopped by trigger signals from the target. |
| **SMA / ribbon / card-edge** | SMA: the small threaded coax connector (Husky `Pos` to the CW313). Ribbon: the flat 20-wire cable. Card-edge: the gold-fingered edge of the target board that pushes into the CW313's J1 socket. |
| `cwtraces` | The Python package that holds the recorded traces for the simulated labs. |

# Appendix C - Where things live

| Item | Location |
|---|---|
| Virtual environment | `~/.cwvenv` |
| ChipWhisperer repository | `~/chipwhisperer` |
| USB rules (installed copy) | `/etc/udev/rules.d/50-newae.rules` (source: `~/chipwhisperer/50-newae.rules`) |

| Item | Location |
|---|---|
| Course notebooks | `~/chipwhisperer/jupyter/courses/sca101/` |
| Firmware used by the labs | `~/chipwhisperer/firmware/mcu/` (for example `basic-passwdcheck`, `simpleserial-base`) |
| Notebook requirements | `~/chipwhisperer/jupyter/requirements.txt` (plus `cwtraces`, installed separately) |
| Lab 2_1A notebooks | `.../sca101/Lab 2_1A - Instruction Power Differences (MAIN).ipynb` and `... Explorer.ipynb` |
| Lab 2_1B Explorer | `.../sca101/Lab 2_1B - Password Bypass Explorer.ipynb` (needs `cwtraces`) |
| Recorded traces for Lab 2_1A | `.../sca101/traces/lab2_1A_trace0.npy` to `trace3.npy` |
| Lab 2_1A firmware working copy | `~/chipwhisperer/firmware/mcu/simpleserial-base-lab2/` (created by the HARDWARE notebook) |

## Notice

This guide accompanies ChipWhisperer course material that references (commercial) training material on ChipWhisperer.io. You can freely execute and use the lab per the open-source license (including in your own courses if you distribute similarly), but you must maintain notice about the source: https://www.ChipWhisperer.io.

Copyright (C) NewAE Technology Inc., 2015-2020. ChipWhisperer is a trademark of NewAE Technology Inc., claimed in all jurisdictions, and registered in at least the United States of America, European Union, and Peoples Republic of China. Tutorials derived from our open-source work must be released under the associated open-source license, and notice of the source must be clearly displayed.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.